

# OpenVAF-r Robustness Campaign

Adversarial testing, mutation fuzzing, and the crash/hang paths found and fixed

Ngspice + OpenVAF Enhancements project

July 2026

# Contents

|                                                     |   |
|-----------------------------------------------------|---|
| OpenVAF-r robustness campaign                       | 2 |
| Summary . . . . .                                   | 3 |
| Method . . . . .                                    | 4 |
| Findings . . . . .                                  | 5 |
| The critical finding: exponential-time nested ?:    | 6 |
| The hardening: parser depth, include cap, array cap | 7 |
| Behaviour-preserving . . . . .                      | 8 |
| Reproducing . . . . .                               | 9 |

# OpenVAF-r robustness campaign

A deep robustness audit of the `openvaf-r` Verilog-A  $\rightarrow$  OSDI compiler: the full production-model corpus,  $\sim 50$  adversarial hand-crafted inputs, and 4,000 mutation-fuzzing iterations, run through a classifier that labels every outcome OK / clean-error / panic / segfault / abort / hang. The compiler came back clean on real work, and the campaign surfaced four ways to make it crash or hang on pathological input — all now fixed.

This is a companion to the [OpenVAF compiler internals](#) guide: that document explains how the compiler works; this one documents how hard it was pushed and what broke.

## Summary

---

|                                 |                                                                 |
|---------------------------------|-----------------------------------------------------------------|
| Production models compiled      | 117 — 0 crashes / hangs / panics                                |
| Adversarial hand-crafted inputs | ~50 — all rejected cleanly, 0 accepted-invalid                  |
| Mutation-fuzzing iterations     | 4,000 — 0 panics, 0 segfaults                                   |
| Crash / hang paths found        | 4 — all fixed ( <a href="#">E-147</a> , <a href="#">E-148</a> ) |

---

The compiler never crashed on random or garbage input — the only failures were specific, structured pathologies, each turned into a clean bounded diagnostic.

## Method

Every input was compiled with a per-file timeout under `RUST_BACKTRACE=1`, and the result classified from the exit signal and output:

- OK — exit 0 (a `.osdi` was produced);
- clean error — nonzero exit with a diagnostic, no panic/crash;
- panic — a Rust panicked at ... (an internal compiler error);
- segfault / abort — killed by SIGSEGV / SIGABRT (e.g. a stack overflow);
- hang — did not finish within the timeout.

Only panic / segfault / abort / hang count as robustness bugs; a clean error on malformed input is the desired outcome.

### Phase A — the real production corpus

Every model in the compact-model test suites (BSIM3/4/6, BSIM-CMG/IMG/SOI/BULK, PSP 102/103/104, HICUM L0/L2, MEXTRAM, VBIC, EKV/EKV3, HiSIM, ASMHMET, r2/r3\_cmc, diode\_cmc, ...) was compiled — 117 files in all.

Result: 117 / 117 compiled with 0 crashes / hangs / panics. (A handful of include-fragment files — function libraries meant to be ``included`, not compiled standalone — report a clean error, as expected.)

### Phase B — adversarial hand-crafted inputs

~50 inputs targeting the parser, preprocessor, and lowering: unterminated constructs (block comment, string, module, parenthesis), malformed numbers (`1e999999`, `4'b2`, `999999999'b1`, `1.2.3`), macro bombs (``define A A`, mutual recursion, an exponential expansion chain), circular / missing includes, duplicate declarations, unicode identifiers and embedded null bytes, and extreme nesting / sizes.

Result: all rejected cleanly; 0 accepted-invalid inputs. Five structured pathologies produced a crash or hang (below); everything else was a clean error. Macro recursion is already capped; ``include` recursion was not.

### Phase C — mutation fuzzing

4,000 iterations, each taking a real model and applying a random mutation — byte flips, truncations, run duplication, delimiter injection, deep-nesting injection — then compiling it.

Result: 0 panics, 0 segfaults across 4,000 iterations. The compiler never crashed on mutated input; a few mutations produced hangs, all instances of the structured pathologies below (macro-expansion / nesting blow-ups).

## Findings

| Severity | Trigger                                                          | Symptom (before)                                                       | Status        |
|----------|------------------------------------------------------------------|------------------------------------------------------------------------|---------------|
| critical | nested ?: chain, depth $\geq$ 30                                 | $O(2^n)$ validation $\rightarrow$ compiler hangs; reachable via macros | fixed (E-147) |
| low      | expression nested ~4k–32k deep (unary / binary / parens / calls) | recursive-descent stack overflow (SIGABRT)                             | fixed (E-148) |
| low      | file that `includes itself                                       | uncapped include recursion $\rightarrow$ stack overflow                | fixed (E-148) |
| low      | absurd array range real x [0:100000000]                          | expanded element-by-element $\rightarrow$ memory exhaustion / hang     | fixed (E-148) |

The three low-severity findings all require input a human or real model would never write. The critical one is different: ~30 nested conditionals is easily produced by macros in a real device model, and it *hangs* the compiler — the worst failure mode.

## The critical finding: exponential-time nested ?:

### Root cause

The body validator visits every expression in `validate_expr`, whose match ends by recursing into the children:

```
_ => ()  
}  
self.parent.body.exprs[expr].walk_child_exprs(|child| self.validate_expr(child))
```

Arms that fully validate their own operands return early to skip that generic walk — the `Call` and `Path` arms both do. The **Select** (ternary) arm did not: it validated `cond`, `then_val` and `else_val` via `validate_condition`, then *fell through* to `walk_child_exprs`, which validated `then_val` and `else_val` a second time. Each ternary therefore validated its branches twice; for a chain of  $N$  nested `?`: this compounds to  $2^n$  validations.

Profiling a hanging compile with a live stack sample confirmed it exactly — an unbounded `validate_expr`  $\rightarrow$  `validate_condition`  $\rightarrow$  `validate_expr`  $\rightarrow$  ... recursion with a doubling call count.

### The fix

Add a `return;` at the end of the `Select` arm, matching the `Call` / `Path` arms. The arm already validates all three children, so the generic walk was pure redundant work — removing it is behaviour-preserving and turns validation from  $O(2^n)$  into  $O(N)$ :

| nesting depth | before               | after          |
|---------------|----------------------|----------------|
| 40            | hang (> 30 s)        | 0.09 s         |
| 160           | $2^{160}$ (hopeless) | 0.11 s         |
| 2000          | —                    | 1.6 s (linear) |

Full write-up: [Enhancement-147](#).

## The hardening: parser depth, include cap, array cap

The three lower-severity findings are pathological-input crashes — a robust compiler should reject them cleanly rather than die. [Enhancement-148](#) adds bounded limits.

Parser expression-depth limit. The Pratt parser recurses through `atom_expr` for every level of nesting *and* builds a left-leaning tree for operator chains — so both an over-deep recursion and an over-deep tree (which later passes recurse over) could overflow the stack. A shared `expr_depth` counter — incremented on each `atom_expr` (recursion) and per operator in the `expr_bp` loop (chain length) — bounds the total expression-tree depth at 1000 and recovers to the next statement boundary.

``include` recursion cap. An `include_depth` counter caps nesting at 64 and emits a new `IncludeRecursionLimit` diagnostic — a self-including file is reported instead of overflowing the stack, mirroring the existing macro-recursion guard.

Array element cap. An array-shaped declaration is flattened into one scalar element per index. A shared `array_elem_count` helper caps the expansion at  $2^{20} \approx 1.05$  M elements and emits a new `ArrayTooLarge` diagnostic, applied at every expansion site: variable arrays, parameter arrays, net/port buses and array function returns (item-tree lowering), and instance arrays — in *both* the item-tree lowering and the elaboration pass, which re-expands `dev s [0:N] ( )` into rendered text.



## Behaviour-preserving

Both fixes are correctness/robustness changes with no effect on valid programs, and that was checked rather than assumed:

- Corpus head-to-head. Every one of the 117 production models compiles to the identical verdict before and after each fix — 0 flips — and BSIM4 (12.6 k lines) still compiles in ~2.3 s.
- Unit tests. The parser, preprocessor, `hir_def`, `hir_ty` and `sim_back` suites pass with no snapshot changes; the build is warning-free.
- Targeted examples. `examples/nested_cond_examples/` (7/7, the exponential fix) and `examples/robustness_examples/` (17/17, the hardening) pin the behaviour: every pathological input now errors cleanly in well under a second, and valid deep-but-reasonable input (nested ternary depth 30, parentheses depth 100, a 100-term sum, small variable and instance arrays) still compiles.

## Reproducing

The campaign is a small Python harness: a classifier that runs `openvaf-r` on an input under a timeout and labels the outcome, plus three drivers — compile the real corpus (Phase A), a battery of hand-crafted adversarial files (Phase B), and a mutation fuzzer that perturbs real models (Phase C). Localizing a hang used two tools: the compiler's own `--dump-j son / --dump-unopt-mir` flags to bisect the pipeline phase, and macOS `sample <pid>` on the live process to name the exact hot function.